

Docklite

Aplicación web para la gestión de
contenedores Docker



Autor: Aimar Merodo Alba

Índice de contenidos

1	Introducción	3
2	Objetivos del proyecto	3
2.1	Objetivo general	3
2.2	Objetivos específicos	3
3	Fases del proyecto	4
3.1	Fase 0 - Preparación del entorno	4
3.2	Fase 1 - Autenticación y gestión de usuarios	4
3.3	Fase 2 - Gestión de contenedores	4
3.4	Fase 3 - Gestión de imágenes, volúmenes y redes	4
3.5	Fase 4 - API completa y modelo invitation-only	5
3.6	Fase 5 - Empaquetado y despliegue	5
3.7	Fase 6 - Desarrollo del Frontend	5
3.8	Fase 7 - Documentación y memoria	6
4	Especificación de requisitos	7
4.1	Actores del sistema	7
4.2	Requisitos funcionales	7
4.3	Requisitos no funcionales	7
5	Análisis	9
5.1	Modelo de negocio	9
5.2	Casos de uso	11
6	Diseño del sistema	17
6.1	Arquitectura general	17
6.2	Diseño de la base de datos	19
6.3	Diseño de la aplicación	20
6.3.1	Pantalla de inicio de sesión	20
6.3.2	Dashboard	21
6.3.3	Gestión de contenedores	22
6.4	Diagrama de navegabilidad	23
7	Implementación	24
7.1	Tecnologías y frameworks	24

7.1.1	Capa de presentación.....	24
7.1.2	Capa lógica – Backend.....	24
7.1.3	Capa de datos.....	25
7.1.4	Infraestructura y despliegue	25
7.1.5	Resumen de las capas	25
7.2	Estructura del código	26
7.2.1	Manejador global de errores.....	27
7.2.2	Capa de presentación: el controlador.....	28
7.2.3	Capa de lógica: el servicio	29
8	Validación y pruebas	31
8.1	Pruebas funcionales	31
8.2	Validación de datos de entrada	31
8.3	Control de acceso y aislamiento (RNF-02)	32
8.4	Análisis estático con SonarQube	32
8.4.1	Excepciones genéricas en operaciones Docker (regla java:S112).....	34
8.4.2	Duplicación de literales (regla java:S1192)	36
8.4.3	Security Hotspots	37
9	Conclusiones.....	39
10	Bibliografía.....	40
10.1	Documentación oficial.....	40
10.2	Especificaciones y estándares	40
10.3	Buenas prácticas y seguridad	41
10.4	Herramientas.....	41
10.5	Recursos consultados durante el desarrollo	41

1 Introducción

Docker se ha convertido en una herramienta ampliamente utilizada para el despliegue de aplicaciones mediante contenedores. Su uso permite aislar aplicaciones y facilitar su distribución en distintos entornos, siendo especialmente común en servidores y plataformas de alojamiento.

Sin embargo, la gestión de Docker se realiza principalmente a través de línea de comandos, lo que puede dificultar su utilización en entornos donde no se dispone de una interfaz gráfica, como servidores remotos o VPS. Además, soluciones como Docker Desktop están orientadas al uso local y no resultan adecuadas para la gestión remota de contenedores en servidores accesibles desde Internet.

Con el objetivo de facilitar el despliegue y la gestión de aplicaciones contenedorizadas en servidores, surge Docklite, una aplicación web que proporciona una interfaz gráfica accesible desde cualquier lugar. Docklite está dirigida a empresas y usuarios individuales que deseen desplegar aplicaciones en su propio servidor y administrarlas de forma sencilla mediante un navegador web, incorporando además un sistema de usuarios que permite gestionar y compartir los recursos de manera controlada.

2 Objetivos del proyecto

2.1 Objetivo general

Desarrollar una aplicación web que permita el despliegue y la gestión de aplicaciones contenedorizadas mediante Docker a través de una interfaz gráfica accesible de forma remota.

2.2 Objetivos específicos

- Permitir el registro por invitación y la autenticación de usuarios en la aplicación.
- Facilitar el despliegue de contenedores Docker mediante una interfaz web.
- Gestionar contenedores Docker, permitiendo su creación, visualización y eliminación.
- Gestionar volúmenes Docker asociados a los contenedores.
- Gestionar redes Docker utilizadas por las aplicaciones desplegadas.
- Garantizar el aislamiento y control de acceso a los recursos Docker mediante un sistema de usuarios.
- Permitir el acceso remoto a la gestión de contenedores desde cualquier ubicación a través de un navegador web en un ordenador.

3 Fases del proyecto

El desarrollo de Docklite se ha organizado siguiendo una **metodología ágil iterativa**, dividiendo el trabajo en fases cortas y entregables que permiten validar el progreso de forma continua. Cada fase se ha definido con un objetivo concreto y un conjunto de funcionalidades asociadas a los requisitos identificados en el apartado anterior.

3.1 Fase 0 - Preparación del entorno

Configuración inicial del repositorio, elección del stack tecnológico y puesta en marcha del entorno de desarrollo. Incluye la instalación de Java 21, Spring Boot, PostgreSQL mediante Docker Compose, la estructura del proyecto Maven y la conexión básica entre el backend y la base de datos. Al finalizar esta fase, el esqueleto del proyecto arranca correctamente y las tablas del modelo (users, invitations, docker_resources, activity_log) se crean automáticamente mediante migraciones Flyway.

3.2 Fase 1 - Autenticación y gestión de usuarios

Implementación del sistema de registro, inicio de sesión y autenticación basada en **JSON Web Tokens**. Se desarrollan las entidades JPA, los repositorios, los endpoints REST de autenticación y el filtro de seguridad que valida el token en cada petición.

Se incorpora también el manejo global de errores mediante `@RestControllerAdvice`, devolviendo respuestas JSON consistentes para todos los códigos de estado (400, 401, 403, 404, 409, 500).

3.3 Fase 2 - Gestión de contenedores

Integración con el motor Docker a través de la librería docker-java. Se implementan los endpoints para listar, crear, arrancar, parar y eliminar contenedores, introduciendo además el mecanismo de **control de propiedad** que filtra los recursos visibles para cada usuario.

Se introduce además la auditoría de acciones mediante la tabla activity_log, que registra cada operación mutante (crear, arrancar, parar, reiniciar y eliminar) junto con el usuario que la realizó.

3.4 Fase 3 - Gestión de imágenes, volúmenes y redes

Extensión del patrón de ownership al resto de recursos Docker. Se añaden los endpoints para gestionar imágenes (con la particularidad de que la eliminación queda reservada al administrador), volúmenes y redes.

Se incluye un mecanismo de pull-on-create: al crear un contenedor con una imagen que no está en el host, el backend la descarga automáticamente desde Docker Hub antes de continuar

3.5 Fase 4 - API completa y modelo invitation-only

Se completa la API con los endpoints de soporte: dashboard agregado (/api/v1/system/dashboard), información del sistema, listado paginado del histórico de actividad y actualización del perfil propio. Se añade documentación interactiva mediante Swagger/OpenAPI y se configura CORS de manera adecuada para el entorno de desarrollo.

En esta fase se realiza también el cambio del modelo de registro: se elimina el registro abierto y se sustituye por un sistema de invitaciones multi-uso emitidas por el administrador. Cada invitación lleva un token aleatorio, un número configurable de usos y una fecha de caducidad. El primer administrador del sistema se crea automáticamente al arrancar el backend a partir de las variables de entorno suministradas por el script de instalación.

3.6 Fase 5 - Empaquetado y despliegue

Se contenedoriza la aplicación generando Dockerfiles multi-stage para el backend (compilación con Maven y ejecución sobre JRE Alpine) y para el frontend (compilación de Angular y servicio mediante nginx). El fichero docker-compose.yml orquesta los tres servicios sobre una red privada, exponiendo al exterior únicamente el frontend.

Asimismo, se desarrolla un script de instalación interactivo (install.sh) que automatiza el despliegue completo. Dicho script detecta el sistema operativo, instala Docker en caso de no estar presente, solicita al administrador los datos de configuración necesarios (dominio o dirección IP, credenciales del primer administrador y puertos), genera de forma aleatoria los secretos para JWT y la base de datos, y arranca la aplicación mediante un único comando. Cuando se opta por la instalación en modo dominio, el script obtiene automáticamente un certificado SSL de Let's Encrypt a través de certbot.

3.7 Fase 6 - Desarrollo del Frontend

Construcción de la interfaz en Angular, incluyendo las pantallas de inicio de sesión, aceptación de invitación, dashboard, gestión de contenedores, imágenes, volúmenes, redes y administración. Se implementa el interceptor HTTP que añade el token JWT a cada petición y los guards que protegen las rutas privadas según el rol del usuario.

3.8 Fase 7 - Documentación y memoria

Redacción de la memoria del proyecto, preparación de la presentación y revisión final del código y la documentación.

Fase	Duración estimada
Fase 0 - Preparación del entorno	1 semana
Fase 1 - Autenticación, manejo de errores y JWT	1 semana
Fase 2 - Contenedores, ownership y auditoría	1-2 semanas
Fase 3 - Imágenes, volúmenes y redes	1-2 semanas
Fase 4 - API completa y modelo invitation-only	1 semana
Fase 5 - Empaquetado y despliegue	1 semana
Fase 6 - Desarrollo del frontend	2 semanas
Fase 7 - Documentación y memoria	A lo largo del proyecto

4 Especificación de requisitos

4.1 Actores del sistema

Docklite cuenta con dos actores principales:

- **Administrador:** usuario con permisos para gestionar todos los contenedores y recursos Docker del sistema, independientemente del usuario propietario.
- **Usuario:** persona autenticada que puede gestionar únicamente los recursos Docker que ha creado dentro de la aplicación.

4.2 Requisitos funcionales

- **RF-01:** El sistema permitirá la creación de nuevos usuarios mediante invitación del administrador.
- **RF-02:** El sistema permitirá la autenticación de usuarios mediante credenciales.
- **RF-03:** El sistema permitirá diferenciar roles entre usuario y administrador.
- **RF-04:** El sistema permitirá a los usuarios normales gestionar únicamente los contenedores Docker creados por ellos.
- **RF-05:** El sistema permitirá al administrador gestionar todos los contenedores y recursos Docker del sistema, independientemente del usuario propietario.
- **RF-06:** El sistema permitirá crear, listar y eliminar contenedores Docker.
- **RF-07:** El sistema permitirá crear, listar y eliminar volúmenes Docker.
- **RF-08:** El sistema permitirá crear, listar y eliminar redes Docker.
- **RF-09:** El sistema permitirá la gestión de contenedores a través de una interfaz web accesible de forma remota.
- **RF-10:** El administrador podrá generar links de invitación multi-uso con caducidad configurable

4.3 Requisitos no funcionales

- **RNF-01 (Seguridad):** El acceso al motor Docker se realizará exclusivamente desde el backend de la aplicación.
- **RNF-02 (Seguridad):** El sistema garantizará el aislamiento de los recursos Docker entre los distintos usuarios.
- **RNF-03 (Usabilidad):** La aplicación será accesible desde un navegador web sin necesidad de software adicional.
- **RNF-04 (Portabilidad):** El sistema deberá poder desplegarse mediante contenedores Docker.
- **RNF-05 (Escalabilidad):** La arquitectura de la aplicación permitirá la incorporación de nuevas funcionalidades en el futuro.

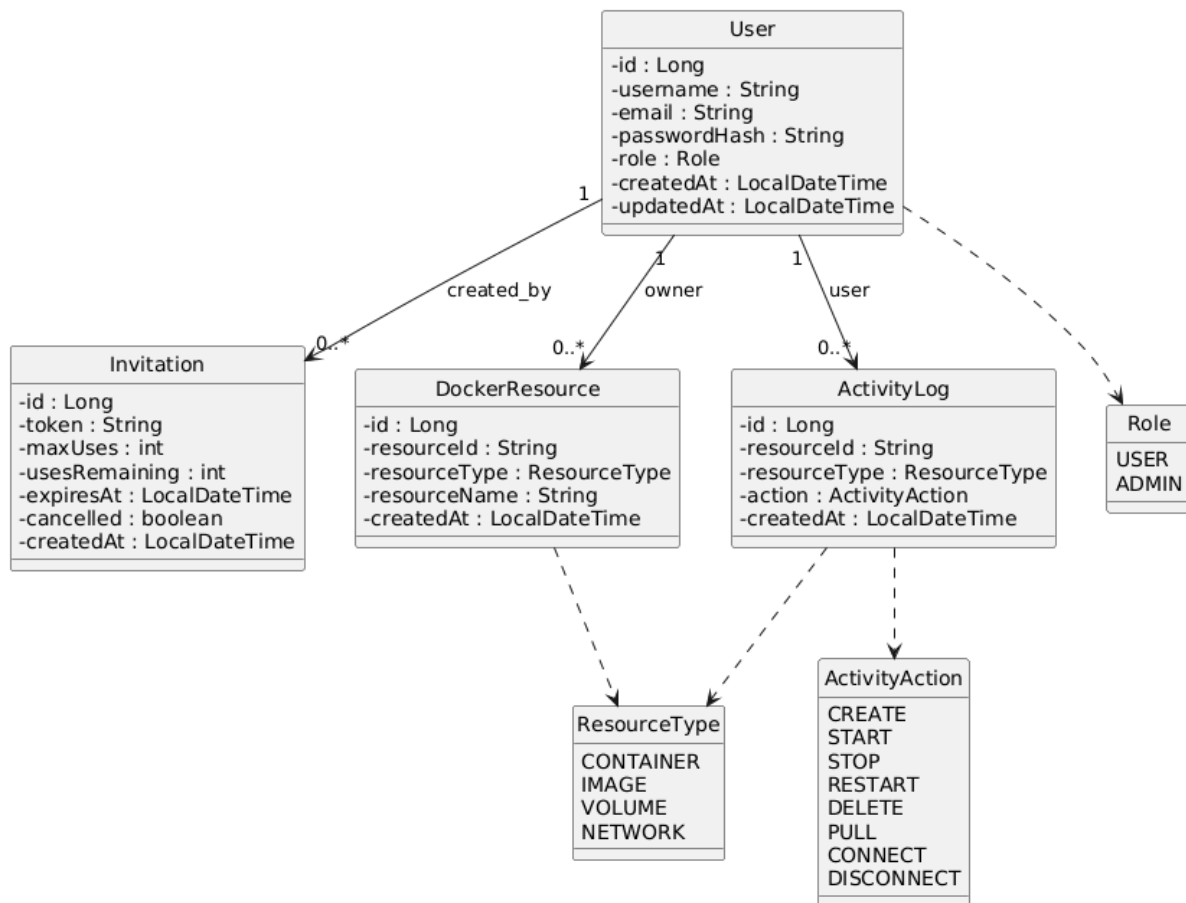
- **RNF-06 (Rendimiento):** Las operaciones básicas de gestión de recursos Docker deberán ejecutarse en un tiempo razonable para el usuario.
- **RNF-07 (Seguridad):** Las contraseñas se almacenarán cifradas mediante un algoritmo robusto (BCrypt) y la autenticación se basará en tokens firmados (JWT) con secretos generados aleatoriamente durante la instalación.
- **RNF-08 (Seguridad):** El sistema soportará HTTPS con certificados gestionados automáticamente por Let's Encrypt cuando el administrador configure un dominio. El backend nunca quedará expuesto directamente al exterior, accediéndose siempre a través del reverse proxy del frontend.
- **RNF-09 (Auditabilidad):** Todas las acciones mutantes sobre los recursos Docker (crear, arrancar, parar, reiniciar, eliminar y conectar a redes) quedarán registradas con identificador del autor, recurso afectado y marca temporal.
- **RNF-10 (Mantenibilidad):** El código fuente cumplirá los criterios del Quality Gate por defecto de SonarQube, manteniendo cero bugs conocidos, cero vulnerabilidades y un nivel de mantenibilidad A.
- **RNF-11 (Compatibilidad):** El sistema soportará despliegues sobre las distribuciones Linux Debian 12 y posteriores, y Ubuntu 22.04 LTS y posteriores. La interfaz web será compatible con los navegadores modernos basados en Chromium, Firefox y Safari.
- **RNF-12 (Disponibilidad y persistencia):** Los contenedores se reiniciarán automáticamente ante fallos y los datos de la base de datos persistirán en un volumen Docker para resistir reinicios o red despliegues.

5 Análisis

El análisis del sistema se ha basado en la identificación de los actores implicados (Usuario y Administrador) y en la descripción detallada de los casos de uso más representativos del sistema. El modelo de negocio queda recogido en el diagrama de clases ya presentado, mientras que el comportamiento dinámico se documenta a continuación mediante fichas de los casos de uso clave.

5.1 Modelo de negocio

El modelo de negocio de Docklite se articula en torno a tres entidades principales: User, Invitation, DockerResource y ActivityLog.



User: representa a un usuario de la aplicación. Contiene credenciales (email y hash de contraseña) y un rol (**USER** o **ADMIN**) que determina sus permisos.

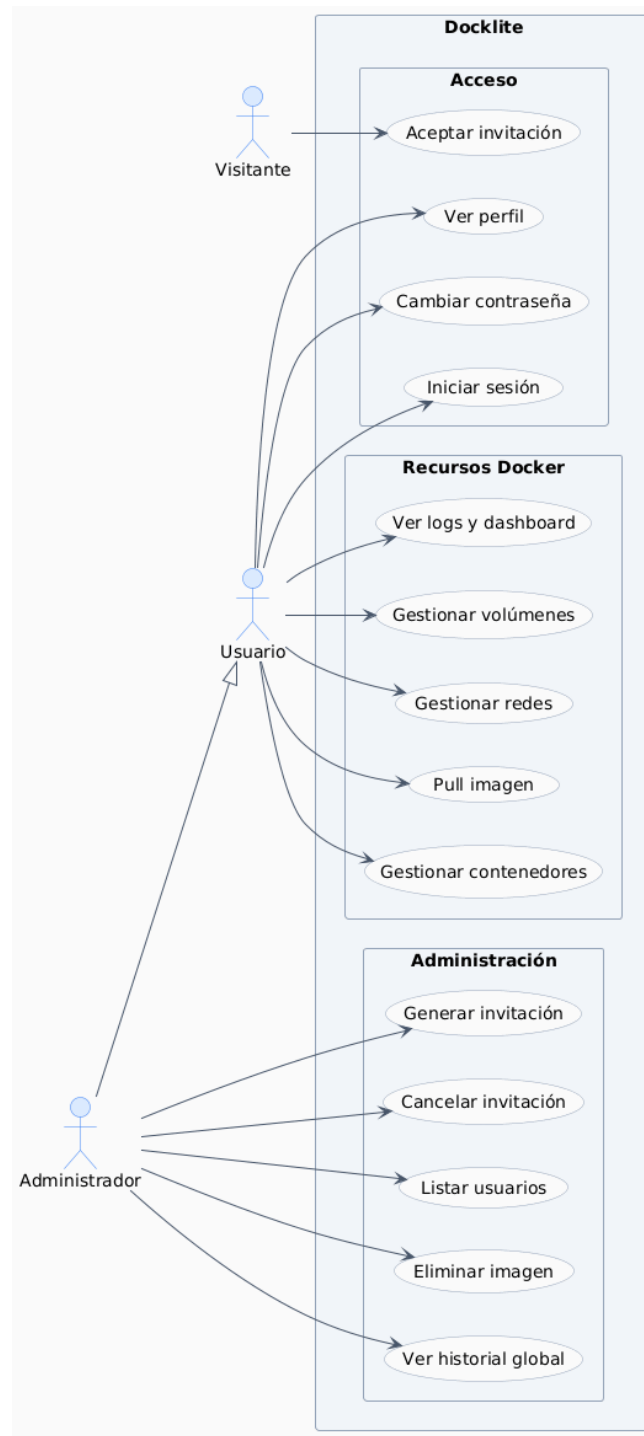
Invitation: representa un link de invitación generado por el administrador para que nuevos usuarios puedan registrarse. Cada invitación tiene un token, un número máximo de usos, una fecha de caducidad y se puede cancelar manualmente. Solo es válida mientras le queden usos, no haya caducado y siga activa.

DockerResource: registra la propiedad de cada recurso Docker creado a través de la aplicación. Su campo `resourceId` guarda el identificador que Docker asigna al recurso, y `resourceType` indica de qué tipo se trata (contenedor, imagen, volumen o red). Un usuario puede poseer varios recursos, y una misma imagen puede tener varios propietarios, ya que Docker mantiene una única copia en disco, aunque la pulleen distintos usuarios.

ActivityLog: almacena el historial de acciones realizadas por cada usuario sobre los recursos Docker, permitiendo auditar el uso del sistema.

5.2 Casos de uso

Se han identificado dos actores en el sistema: el **Usuario** estándar, que gestiona únicamente sus propios recursos, y el **Administrador**, que hereda todos los casos del usuario y añade capacidades de gestión global.



El sistema cuenta con tres actores:

- **Visitante:** persona no autenticada que accede al sistema mediante un link de invitación generado por un administrador.
- **Usuario:** persona autenticada con rol USER, que gestiona únicamente los recursos Docker creados por ella.
- **Administrador:** hereda todas las capacidades del Usuario y añade la gestión global del sistema (invitaciones, gestión de cualquier recurso y eliminación de imágenes).

A continuación, se describen los casos de uso más representativos mediante fichas detalladas.

Identificador	CU-01
Nombre	Aceptar invitación
Actor principal	Visitante
Precondiciones	- Existe una invitación válida (no cancelada, no caducada, con usos restantes > 0). - El visitante dispone del link compartido por un administrador.
Postcondiciones	- Se crea un nuevo usuario con rol USER. - El contador "usesRemaining" de la invitación se decrementa en 1. - El visitante recibe un JWT y queda autenticado.
Flujo principal	1. El visitante abre el link de invitación. 2. El sistema valida que la invitación existe, no está cancelada, no ha caducado y le quedan usos disponibles. 3. El sistema muestra el formulario de registro. 4. El visitante introduce username, email y contraseña. 5. El sistema valida que email y username no existan ya en la base de datos. 6. El sistema crea el usuario con la contraseña cifrada mediante BCrypt. 7. El sistema decrementa el contador de usos restantes de la invitación. 8. El sistema genera un JWT y lo devuelve junto con los datos del nuevo usuario.
Flujos alternativos	2a. La invitación no existe, está cancelada, caducada o agotada → 404 Not Found. 5a. El email ya está registrado → 409 Conflict. 5b. El username ya está registrado → 409 Conflict. 4a. Algún campo no cumple las validaciones (formato, longitud) → 400 Bad Request con detalle de los campos erróneos.
Requisitos asociados	RF-01, RF-10
Prioridad	Alta
Frecuencia de uso	Baja — solo al incorporar nuevos usuarios.

Identificador	CU-03
Nombre	Generar invitación
Actor principal	Administrador
Precondiciones	- El administrador tiene una sesión iniciada con rol ADMIN.
Postcondiciones	- Se crea una nueva fila en la tabla invitations con un token único, su límite de usos y su fecha de caducidad. - El sistema devuelve la URL completa lista para compartir.
Flujo principal	1. El administrador accede al panel de gestión de invitaciones. 2. Indica el número máximo de usos y los días de caducidad (ambos opcionales — si los omite, se aplican los defaults: 1 uso y 7 días). 3. El sistema genera un token aleatorio (UUID). 4. Crea la invitación en base de datos asociada al administrador que la generó. 5. Devuelve la URL completa (https://<dominio>/invite/<token>) para que el administrador la comparta.
Flujos alternativos	2a. Los valores indicados son inválidos (maxUses < 1 o expiresInDays < 1) → 400 Bad Request con detalle del campo. 1a. El usuario no tiene rol ADMIN → 403 Forbidden. 6a. El administrador puede cancelar la invitación posteriormente → la invitación queda invalidada (CU complementario, no detallado).
Requisitos asociados	RF-10
Prioridad	Alta
Frecuencia de uso	Baja — solo al incorporar nuevos usuarios.

Identificador	CU-04
Nombre	Crear contenedor
Actor principal	Usuario / Administrador
Precondiciones	- El actor tiene una sesión iniciada. - El motor Docker del host está accesible.
Postcondiciones	- Se crea un nuevo contenedor en Docker. - Se registra una fila en docker_resources asociando el contenedor al usuario que lo creó. - Si la imagen no estaba en local, queda descargada y registrada como propiedad del usuario. - Se inserta una entrada CREATE en activity_log (y otra START si autoStart era true).
Flujo principal	1. El usuario indica imagen, nombre del contenedor (opcional) y si debe arrancar automáticamente (autoStart). 2. El sistema comprueba si la imagen existe en el host. Si no, hace pull desde Docker Hub antes de continuar. 3. El sistema crea el contenedor en Docker. 4. Registra la propiedad en docker_resources con el usuario actual como dueño. 5. Si autoStart es true, arranca el contenedor. 6. Registra la(s) acción(es) en activity_log. 7. Devuelve los datos del contenedor recién creado (id, nombre, estado).
Flujos alternativos	1a. El campo "image" está vacío → 400 Bad Request. 2a. La imagen no existe en Docker Hub → 404 Not Found. 3a. El nombre indicado ya está siendo usado por otro contenedor del host → error de Docker propagado al cliente.
Requisitos asociados	RF-04, RF-05, RF-06, RF-09
Prioridad	Alta
Frecuencia de uso	Alta — caso de uso central del sistema.

Identificador	CU-06
Nombre	Eliminar imagen
Actor principal	Administrador (exclusivo)
Precondiciones	- El administrador tiene sesión iniciada con rol ADMIN. - La imagen existe en el host Docker.
Postcondiciones	- La imagen queda eliminada del host Docker. - Se borran TODAS las filas de docker_resources que referencian esa imagen, sin importar el propietario. - Se inserta una entrada DELETE en activity_log atribuida al administrador.
Flujo principal	1. El administrador accede al listado de imágenes y solicita la eliminación de una de ellas. 2. El sistema verifica que el actor tiene rol ADMIN. 3. Solicita al motor Docker la eliminación de la imagen. 4. Borra de docker_resources todas las filas que referencian esa imagen, independientemente del usuario propietario. 5. Registra la acción en activity_log. 6. Devuelve confirmación de éxito.
Flujos alternativos	2a. El usuario no tiene rol ADMIN → 403 Forbidden. 3a. La imagen no existe en el host → 404 Not Found. 3b. La imagen está en uso por uno o varios contenedores → error de Docker propagado al cliente.
Requisitos asociados	RF-05
Prioridad	Media
Frecuencia de uso	Baja — limpieza puntual del host por parte del administrador.

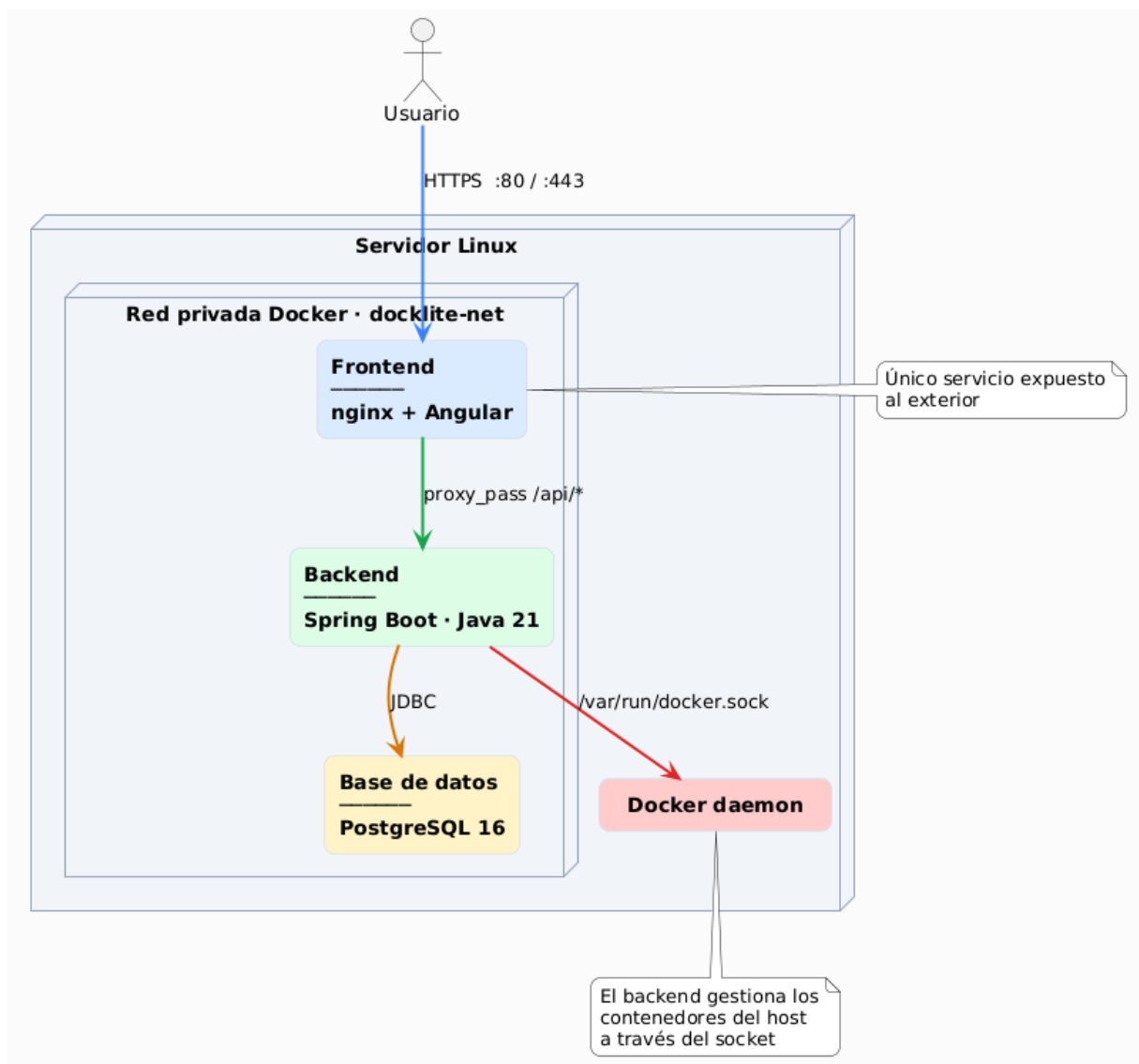
La eliminación se restringe al administrador porque Docker mantiene una única copia de cada imagen en disco, aunque la hayan descargado varios usuarios. Si se permitiera a un usuario corriente borrarla, podría afectar al resto.

6 Diseño del sistema

6.1 Arquitectura general

Docklite tiene una arquitectura **cliente – servidor** en tres capas: frontend, backend y base de datos, todas desplegadas como contenedores Docker sobre una misma red privada. La separación facilita el mantenimiento y permite escalar el sistema en el futuro.

El sistema se compone de cinco bloques:



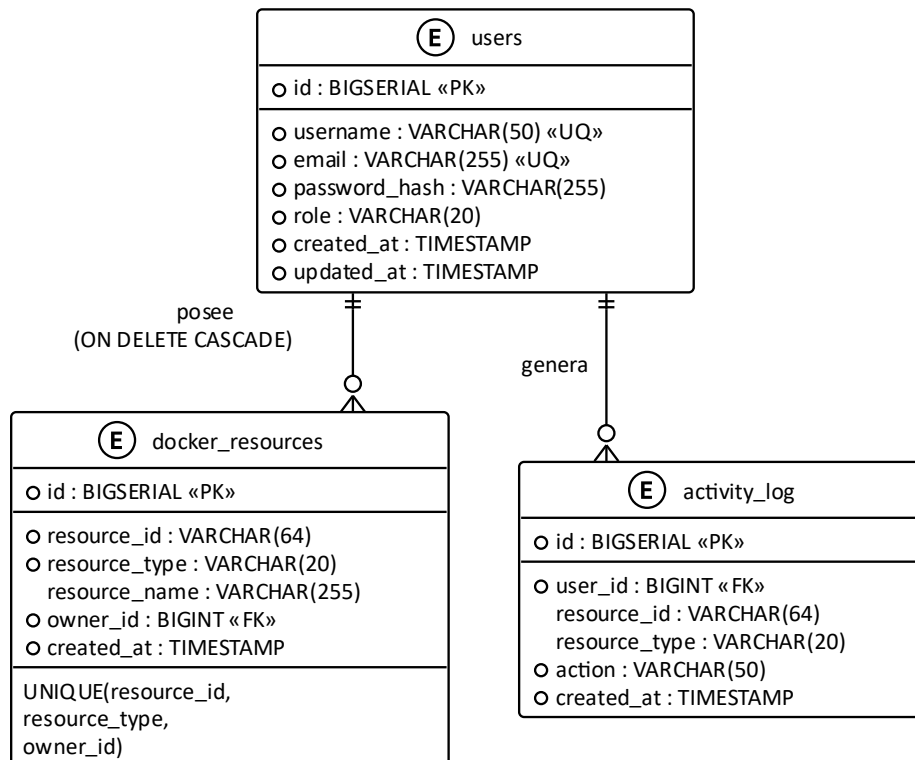
- **Cliente:** navegador web, único punto de entrada
- **Frontend:** aplicación Angular (Single Page Application) servida por nginx. Además de la SPA, nginx actúa como reverse proxy redirigiendo todas las peticiones que empiezan por /api hacia el backend a través de la red interna de Docker. Así el navegador siempre habla con un único origen (modelo same-origin), evitando la necesidad de CORS en producción.
- **Backend:** aplicación Java con Spring boot, organizada en capas (controladores -> servicios -> repositorios). Spring Security valida el JWT en cada petición mediante un filtro personalizado y la librería docker-java es el único componente autorizado a comunicarse con el motor Docker. Al arrancar por primera vez crea automáticamente la cuenta de administrador a partir de las variables suministradas por el script de instalación.
- **Base de datos:** PostgreSQL con cuatro tablas (users, invitations, docker_resources y activity_log), gestionadas mediante migraciones versionadas con Flyway
- **Docker Engine:** recibe las órdenes del backend a través del socket local del demonio. El navegador nunca accede a él directamente.

Solo el Frontend está expuesto al exterior; backend y base de datos viven en la red interna de Docker, inaccesibles desde fuera. El control de propiedad se aplica en la capa de servicios: cuando un usuario lista recursos, el backend filtra la respuesta del demonio para devolver solo los registrados a su nombre en docker_resources, salvo que tenga rol **ADMIN**, en cuyo caso ve todos. Este mecanismo garantiza el aislamiento entre usuarios.

6.2 Diseño de la base de datos

La base de datos de Docklite se ha diseñado con tres tablas que cubren las necesidades del modelo: gestión de usuarios, control de propiedad sobre los recursos Docker y registro de auditoría.

Diagrama Entidad-Relación — DockLite



- **users:** datos de los usuarios registrados. El rol (USER o ADMIN) se guarda como texto por legibilidad.
- **docker_resources:** asocia cada recurso Docker con su propietario. La restricción `UNIQUE(resource_id, resource_type, owner_id)` permite que varios usuarios "posean" la misma imagen (Docker guarda una única copia en disco) pero impide duplicados por usuario. Usa `ON DELETE CASCADE` para limpiar recursos al borrar un usuario.
- **activity_log:** registra las acciones mutantes para auditoría. Sin CASCADE, para conservar el histórico, aunque se borre el usuario.

El esquema se gestiona con migraciones versionadas en **Flyway**, aplicadas automáticamente al arrancar el backend.

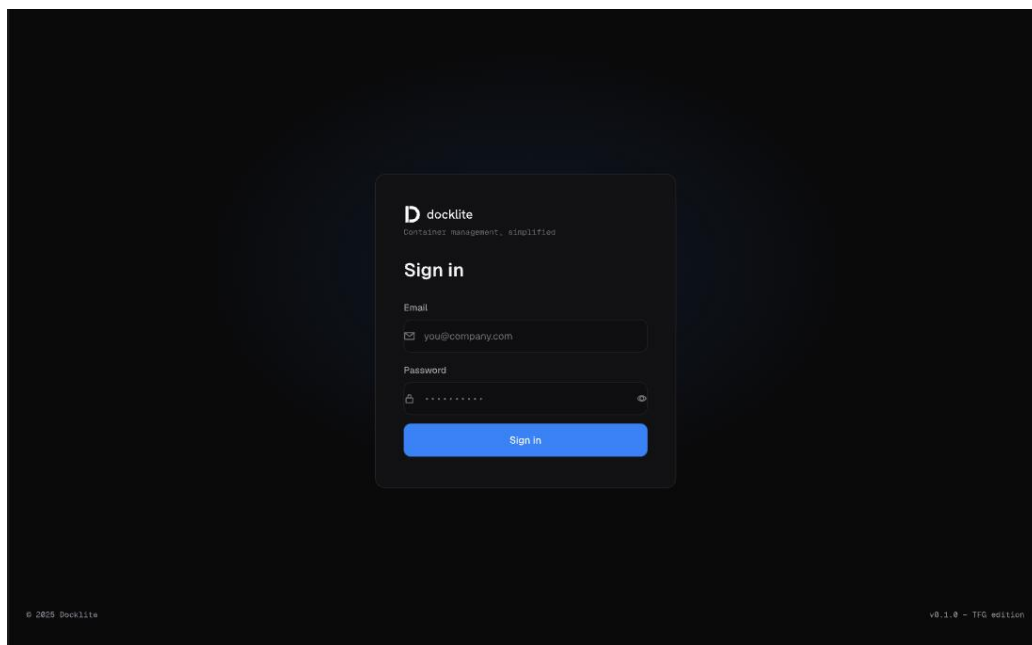
6.3 Diseño de la aplicación

El diseño de la interfaz de Docklite sigue una filosofía minimalista centrada en la productividad: el usuario debe poder localizar y gestionar sus recursos Docker con el menor número posible de clics. La paleta de colores y la tipografía se han elegido para que la interfaz resulte sobria y legible.

A continuación, se muestran los mockups de las pantallas principales de la aplicación. Es importante señalar que estos mockups son orientativos: representan la idea inicial sobre cómo se estructura cada pantalla y los elementos que debe contener.

6.3.1 Pantalla de inicio de sesión

La puerta de entrada al sistema. Permite a un usuario ya registrado identificarse con su email y contraseña. Si las credenciales son correctas, el sistema devuelve un JWT y redirige al dashboard. En caso contrario se muestra un mensaje genérico ("credenciales inválidas") sin distinguir si falla el email o la contraseña, para no revelar información a posibles atacantes.



6.3.2 Dashboard

Pantalla principal tras autenticarse. Muestra un resumen agregado del estado del sistema: número total de contenedores y cuántos están en ejecución, total de imágenes, redes y volúmenes a los que tiene acceso el usuario. Sirve como punto de partida para navegar al detalle de cada recurso. Los datos respetan el patrón de propiedad: un usuario corriente solo ve los contadores de sus propios recursos, mientras que el administrador ve los del sistema completo.

The screenshot displays the Docklite Dashboard interface. On the left is a sidebar with navigation links: Dashboard (selected), Containers, Images, and Volumes. The main content area is titled 'Dashboard' with the subtitle 'Overview of your Docker host'. It features four summary cards: Containers (12 total, 8 running, 3 stopped, 1 paused), Images (24 total, 4.2 GB used), Volumes (7 total, 1.6 GB used), and Networks (3 total, 2 custom, 1 bridge). Below these is a 'Recent containers' section with a table listing containers and their details.

STATUS	NAME	IMAGE	CREATED	
●	nginx-prod	nginx:1.25-alpine	2 hours ago	□
●	postgres-prod	postgres:16-alpine	5 hours ago	□
●	redis-cache	redis:7-alpine	yesterday	□
●	api-gateway	node:20-alpine	2 days ago	□
●	grafana-dev	grafana/grafana:10.4	3 days ago	□

At the bottom left, a user profile card shows 'AM Almar Mero'.

6.3.3 Gestión de contenedores

Listado de los contenedores del usuario con sus principales datos (nombre, imagen, estado y acciones disponibles). Desde aquí pueden realizarse las operaciones más habituales del ciclo de vida de un contenedor: crearlo a partir de una imagen, arrancarlo, pararlo, reiniciarlo, consultar sus logs y eliminarlo. El listado respeta el patrón de propiedad: un usuario corriente solo ve sus contenedores; el administrador ve todos los del host.

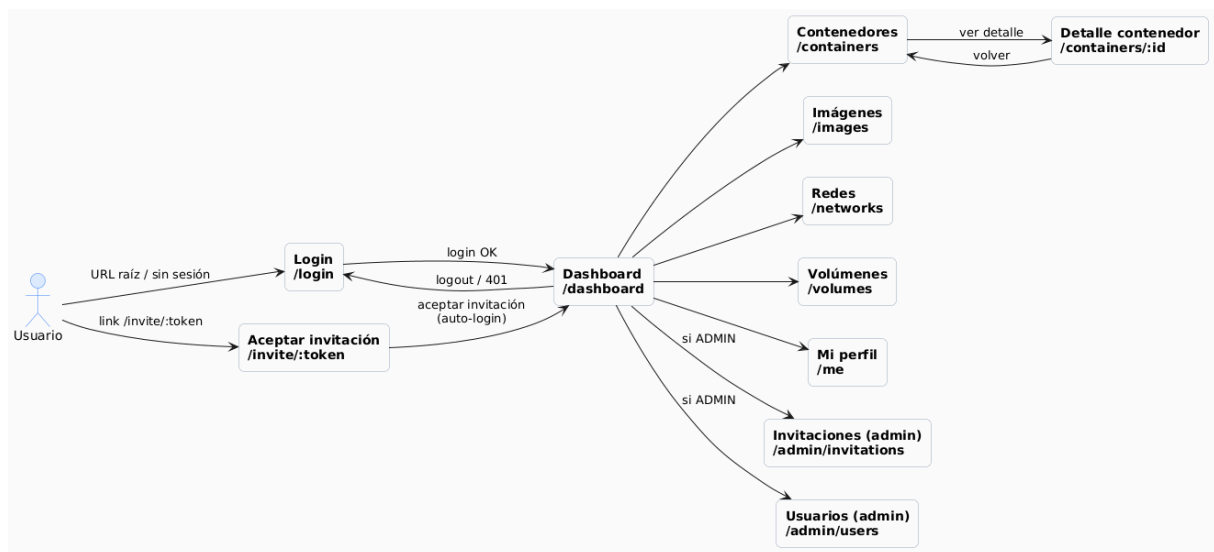
STATUS	NAME	IMAGE	PORTS	CREATED	UPTIME
Running	nginx-prod	nginx:1.25-alpine	8080:80, 443:443	2 hours ago	2h 14m
Running	postgres-prod	postgres:10-alpine	5432:5432	5 hours ago	5h 40m
Running	api-gateway	node:20-alpine	3000:3000	2 days ago	—
Running	mongo-test	mongo:7	—	4 days ago	—
Running	redis-cache	redis:7-alpine	6379:6379	yesterday	18h 32m
Running	traefik	traefik:v3	80, 443, 8080	1 week ago	7d 12h
Failed	worker-failed	python:3.12	—	2 weeks ago	—

6.4 Diagrama de navegabilidad

El diagrama de navegabilidad recoge las rutas del SPA y las transiciones entre ellas. Las pantallas se agrupan en tres zonas según su nivel de acceso:

- Acceso público: login y aceptación de invitación, accesibles sin autenticación.
- Zona de usuario: protegida por un AuthGuard que redirige a /login si no hay token JWT válido.
- Zona admin: protegida adicionalmente por un RoleGuard que solo permite el acceso a usuarios con rol ADMIN.

Tras un login correcto o tras aceptar una invitación, el usuario entra al dashboard, que actúa como hub principal y permite navegar al resto de pantallas. Si una petición devuelve 401 o el usuario hace logout, el sistema lo redirige de vuelta a /login.



7 Implementación

7.1 Tecnologías y frameworks

La elección de las tecnologías de Docklite se ha realizado priorizando la madurez del ecosistema, la compatibilidad con estándares abiertos y la productividad durante el desarrollo.

7.1.1 Capa de presentación

Angular (v21) Framework para Single Page Applications con sistema de componentes, enrutado y cliente HTTP con interceptores, utilizados para añadir el JWT a cada petición. typescript aporta tipado estático y mejor mantenimiento.

7.1.2 Capa lógica – Backend

La elección de Java con Spring Boot 4 como tecnología del backend se realizó tras descartar la opción inicial de Node.js con Express. La decisión se tomó por varios motivos:

- **Madurez del ecosistema:** Spring Security ofrece de serie un sistema de filtros, autenticación, autorización por roles y gestión de excepciones consistente; replicar lo mismo en Express requiere componer media docena de librerías de terceros con calidades desiguales.
- **Tipado estático:** Java permite que muchos errores (DTOs mal formados, payloads incoherentes, llamadas con tipos incorrectos) se detecten en tiempo de compilación. En un proyecto centrado en seguridad y aislamiento entre usuarios, evitar fallos en runtime es crítico.
- **docker-java:** el cliente oficial de Docker para Java es maduro y está actualizado, mientras que las librerías equivalentes en Node.js (dockerode y derivados) son envoltorios menos pulidos sobre el endpoint HTTP.

Spring Boot 4 / Java 21. Framework de referencia en Java. Su autoconfiguración reduce el código repetitivo y acelera el desarrollo. Java 21 aporta mejoras en rendimiento.

Spring Security + jjwt. Gestiona la autorización por roles y el filtro de seguridad. La librería jjwt se encarga de la creación y validación de los tokens JWT.

docker-java. Cliente Java oficial para comunicarse con el daemon Docker mediante su API REST. Evita tener que ejecutar comandos del sistema y facilita las pruebas.

7.1.3 Capa de datos

PostgreSQL 16. SGBD elegido por su madurez, su soporte de constraints complejas (necesarias para el UNIQUE compuesto de `docker_resources`) y su buena integración con Spring Data JPA.

Spring Data JPA + Hibernate. Implementación del estándar JPA. Spring Data genera automáticamente los repositorios a partir de interfaces, reduciendo el código necesario para acceder a los datos.

Flyway. Gestiona las migraciones de la base de datos mediante ficheros `.sql` versionados, aplicados automáticamente al arrancar el backend. Garantiza que cualquier entorno tenga el mismo esquema.

Lombok. Genera getters, setters, constructores y builders mediante anotaciones.

7.1.4 Infraestructura y despliegue

Docker Compose + Maven. Docker Compose levanta la base de datos y el frontend con un único comando, garantizando un entorno reproducible y mantenible. Maven gestiona las dependencias y el ciclo de construcción del backend.

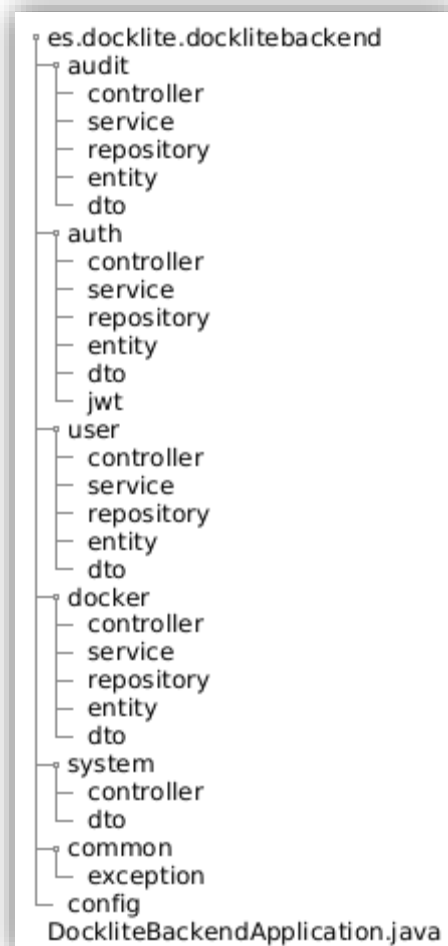
7.1.5 Resumen de las capas

Capa	Tecnología	Justificación
Presentación	Angular v21	SPA con tipado estático e interceptor JWT
Backend	Spring Boot 4 / Java 21	Autoconfiguración y ecosistema maduro
Seguridad	Spring Security + jjwt	Autenticación JWT y control por roles
Datos	Spring Data JPA + Hibernate	Repositorios autogenerados
BBDD	PostgreSQL 16	Robustez y soporte de constraints avanzadas
Migraciones	Flyway	Control de versiones del esquema
Docker	docker-java	Cliente oficial para el daemon
Despliegue	Docker Compose + Maven	Entorno reproducible

7.2 Estructura del código

El código se organiza siguiendo un esquema de **package-by-feature**: cada área funcional (autenticación, usuarios, contenedores, auditoría) vive en su propio paquete con sus controladores, servicios, repositorios y DTOs. Es preferible al clásico package-by-layer porque mantiene junto todo lo relacionado con una misma funcionalidad y evita saltar entre paquetes para tocar una sola característica.

Dentro de cada módulo se aplica una arquitectura clásica en tres capas: el controlador recibe la petición HTTP y delega, el servicio contiene la lógica de negocio y el repositorio se ocupa de la persistencia. Los DTOs se implementan como record de Java 21 para garantizar inmutabilidad sin código repetitivo. Aparte existen unos pocos paquetes transversales (common.exception, config, audit) con componentes compartidos por todo el proyecto.



7.2.1 Manejador global de errores

El primer fragmento corresponde al manejador global de excepciones, anotado con **@RestControllerAdvice**. Centraliza la traducción de excepciones a respuestas HTTP, de modo que ningún controlador necesita envolver su lógica en bloques try/catch: cualquier excepción lanzada desde un servicio (credenciales inválidas, recurso no encontrado, cuenta bloqueada, fallo de Docker, etc.) se intercepta aquí y se convierte en un `ErrorResponse` con un código y un mensaje coherentes en toda la API.

```
@Slf4j @Almar *
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler({EmailAlreadyExistsException.class, UsernameAlreadyExistsException.class}) @Almar
    public ResponseEntity<ErrorResponse> handleConflict(RuntimeException ex) {
        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(ErrorResponse.of(HttpStatus.CONFLICT, ex.getMessage()));
    }

    @ExceptionHandler(InvalidCredentialsException.class) @Almar
    public ResponseEntity<ErrorResponse> handleUnauthorized(InvalidCredentialsException ex) {
        return ResponseEntity
            .status(HttpStatus.UNAUTHORIZED)
            .body(ErrorResponse.of(HttpStatus.UNAUTHORIZED, ex.getMessage()));
    }

    @ExceptionHandler(AccessDeniedException.class) @Almar
    public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex) {
        return ResponseEntity
            .status(HttpStatus.FORBIDDEN)
            .body(ErrorResponse.of(HttpStatus.FORBIDDEN, SecurityMessages.ACCESS_DENIED));
    }

    @ExceptionHandler(AuthenticationException.class) @Almar
    public ResponseEntity<ErrorResponse> handleAuthentication(AuthenticationException ex) {
        return ResponseEntity
            .status(HttpStatus.UNAUTHORIZED)
            .body(ErrorResponse.of(HttpStatus.UNAUTHORIZED, message: "Authentication required"));
    }

    @ExceptionHandler(NoResourceFoundException.class) @Almar
    public ResponseEntity<ErrorResponse> handleNoResource(NoResourceFoundException ex) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ErrorResponse.of(HttpStatus.NOT_FOUND, message: "Endpoint not found"));
    }
}
```

7.2.2 Capa de presentación: el controlador

Este fragmento corresponde a un controlador REST (**AuthController**), pieza típica de la capa de presentación. Anotado con `@RestController` y `@RequestMapping`, expone los endpoints públicos de autenticación (`/login`, `/refresh`, `/logout`) y se limita a recibir la petición HTTP, validar el cuerpo con `@Valid` y delegar inmediatamente en el servicio. No contiene lógica de negocio: esa decisión deliberada mantiene los controladores ligeros y facilita reemplazarlos o probarlos de forma aislada.

```
package es.docklite.docklitebackend.auth.controller;

import ...

@RestController @Almar
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;

    @PostMapping("/login") @Almar
    public AuthResponse login(@Valid @RequestBody LoginRequest req) { return authService.login(req); }

    @PostMapping("/refresh") new *
    public AuthResponse refresh(@Valid @RequestBody RefreshTokenRequest req) {
        return authService.refresh(req.refreshToken());
    }

    @PostMapping("/logout") new *
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void logout(@Valid @RequestBody RefreshTokenRequest req) { authService.logout(req.refreshToken()); }
}
```

7.2.3 Capa de lógica: el servicio

Este fragmento corresponde a un servicio de negocio (ContainerService.create), que muestra el rol central de esta capa: orquestar la lógica que el controlador delega. En unas pocas decenas de líneas concentra todo lo que conlleva crear un contenedor: garantizar que la imagen está disponible localmente (descargándola si no), traducir el DTO de entrada en un HostConfig de Docker con red, volúmenes, variables de entorno, puertos, política de reinicio y límites de CPU/memoria, validar que el usuario tiene permiso sobre los recursos referenciados, registrar la propiedad del nuevo contenedor en la base de datos y dejar trazado el evento en el log de auditoría.

Esta concentración de responsabilidades es deliberada: el servicio es el único lugar del proyecto donde se orquesta una operación con el daemon Docker, lo que garantiza que las reglas de propiedad y auditoría se aplican de forma consistente en todo el sistema.

```
public ContainerDto create(CreateContainerRequest req, User currentUser) {
    ensureImageAvailable(req.image(), currentUser);

    CreateContainerCmd cmd = dockerClient.createContainerCmd(req.image());
    if (req.name() != null && !req.name().isBlank()) {
        cmd = cmd.withName(req.name());
    }

    HostConfig hostConfig = HostConfig.newHostConfig();
    boolean hostConfigUsed = false;

    if (req.networkId() != null && !req.networkId().isBlank()) {
        checkNetworkAccess(req.networkId(), currentUser);
        hostConfig = hostConfig.withNetworkMode(req.networkId());
        hostConfigUsed = true;
    }

    if (req.volumes() != null && !req.volumes().isEmpty()) {
        List<Bind> binds = new ArrayList<>();
        for (VolumeMount mount : req.volumes()) {
            if (!ownershipService.hasAccess(mount.volumeName(), ResourceType.VOLUME, currentUser)) {
                throw new AccessDeniedException(SecurityMessages.ACCESS_DENIED);
            }
            Volume target = new Volume(mount.containerPath());
            AccessMode mode = mount.readOnly() ? AccessMode.ro : AccessMode.rw;
            binds.add(new Bind(mount.volumeName(), target, mode));
        }
        hostConfig = hostConfig.withBinds(binds);
        hostConfigUsed = true;
    }

    if (req.env() != null && !req.env().isEmpty()) {
        List<String> envList = req.env().stream()
            .map(e -> e.name() + "=" + (e.value() == null ? "" : e.value()))
            .toList();
        cmd = cmd.withEnv(envList);
    }
}
```

```
if (req.ports() != null && !req.ports().isEmpty()) {
    List<ExposedPort> exposedPorts = new ArrayList<>();
    Ports portBindings = new Ports();
    for (PortMapping pm : req.ports()) {
        InternetProtocol proto = pm.isUdp() ? InternetProtocol.UDP : InternetProtocol.TCP;
        ExposedPort exposed = new ExposedPort(pm.containerPort(), proto);
        exposedPorts.add(exposed);
        portBindings.bind(exposed, Ports.Binding.bindPort(pm.hostPort()));
    }
    cmd = cmd.withExposedPorts(exposedPorts);
    hostConfig = hostConfig.withPortBindings(portBindings);
    hostConfigUsed = true;
}

if (req.restartPolicy() != null && !req.restartPolicy().isBlank()) {
    hostConfig = hostConfig.withRestartPolicy(buildRestartPolicy(req.restartPolicy()));
    hostConfigUsed = true;
}

if (req.memoryMb() != null) {
    hostConfig = hostConfig.withMemory(req.memoryMb().longValue() * 1024L * 1024L);
    hostConfigUsed = true;
}

if (req.cpus() != null) {
    hostConfig = hostConfig.withNanoCPUs((long) (req.cpus() * 1_000_000_000L));
    hostConfigUsed = true;
}

if (hostConfigUsed) {
    cmd = cmd.withHostConfig(hostConfig);
}

CreateContainerResponse response = cmd.exec();
String containerId = response.getId();

ownershipService.register(containerId, ResourceType.CONTAINER, req.name(), currentUser.getId());
```

```
ownershipService.register(containerId, ResourceType.CONTAINER, req.name(), currentUser.getId());
activityLogService.log(currentUser.getId(), containerId, ResourceType.CONTAINER, ActivityAction.CREATE);

if (req.autoStart()) {
    dockerClient.startContainerCmd(containerId).exec();
    activityLogService.log(currentUser.getId(), containerId, ResourceType.CONTAINER, ActivityAction.START);
}

return inspect(containerId, currentUser);
}
```

8 Validación y pruebas

La validación del backend de Docklite combina cuatro enfoques complementarios: pruebas funcionales manuales sobre todos los endpoints, validación automática de los datos de entrada mediante **Bean Validation**, pruebas específicas de control de acceso y aislamiento entre usuarios, y análisis estático del código fuente con SonarQube.

8.1 Pruebas funcionales

Las pruebas funcionales se han llevado a cabo mediante el cliente HTTP integrado en IntelliJ IDEA, utilizando ficheros **.http** versionados en el repositorio que documentan cada caso. Esto permite reproducir cualquier prueba con un único clic y compartirla junto con el código fuente.

Se han organizado en tres ficheros temáticos:

- **flow-admin.http**: ciclo completo de un administrador (login, gestión de invitaciones, dashboard, CRUD de contenedores, imágenes, redes y volúmenes).
- **flow-user.http**: flujo de un usuario corriente, incluyendo la aceptación de una invitación, la creación y el ciclo de vida de sus propios recursos, así como los casos en los que debería ser rechazado por falta de permisos.
- **validations.http**: cobertura sistemática de todos los códigos de error (400, 401, 403, 404, 409 y 500), incluyendo validaciones de formato, autenticación, autorización y conflictos de unicidad.

En total se han probado de forma manual los más de 30 endpoints de la API con sus principales caminos felices y de error.

8.2 Validación de datos de entrada

La capa de presentación valida automáticamente todos los DTOs de entrada mediante Jakarta Bean Validation. Las anotaciones aplicadas incluyen `@NotBlank`, `@Email`, `@Size` y `@Min`, y cualquier violación se traduce en una respuesta 400 Bad Request con el detalle de los campos que han fallado.

Ejemplos validados:

- Aceptar una invitación con un email mal formado da error 400 con `{"email": "debe ser una dirección de correo electrónico con formato correcto"}`.
- Crear un contenedor con campo `image` vacío da error 400 con detalle del campo obligatorio.

- Cambiar la contraseña con menos de 6 caracteres da error 400.

El manejador global de excepciones (@RestControllerAdvice) garantiza que la estructura JSON de error sea consistente para toda la API.

8.3 Control de acceso y aislamiento (RNF-02)

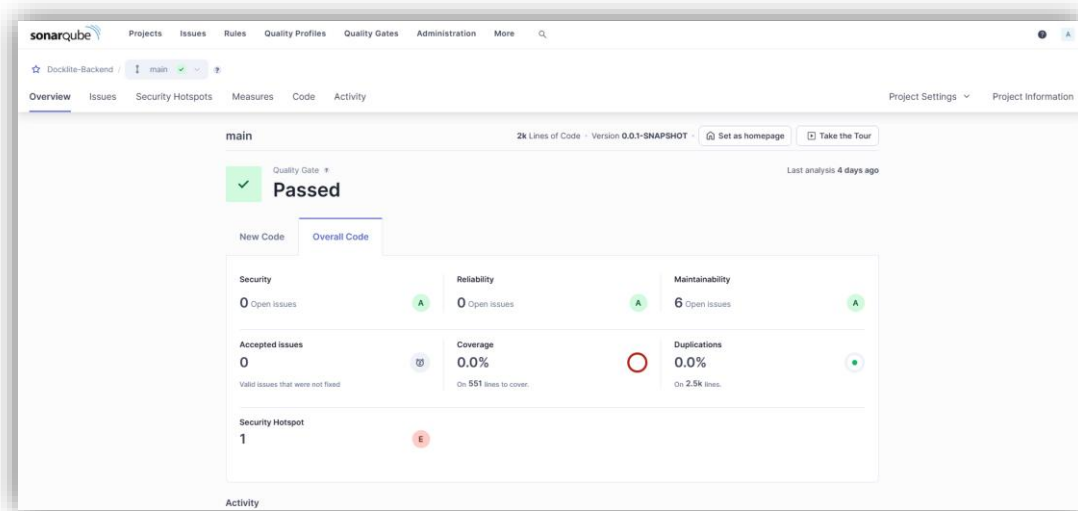
Se han realizado pruebas específicas para verificar el cumplimiento del requisito no funcional de aislamiento. Para ello se han creado dos usuarios de prueba (UserA y UserB) y un administrador, y se han ejecutado los siguientes escenarios:

- UserA crea un contenedor → UserB no lo ve al listar.
- UserB intenta inspeccionar, parar o eliminar el contenedor de UserA → en todos los casos el sistema responde 403 Forbidden.
- UserA intenta eliminar una imagen → 403 Forbidden (operación reservada a administradores).
- UserB intenta acceder a /api/v1/admin/invitations → 403 Forbidden.
- Cualquier endpoint privado sin token JWT válido → 401 Unauthorized.
- El administrador puede operar sobre cualquier recurso, sin restricción.

Estas pruebas confirman que el patrón de propiedad implementado en la capa de servicios es eficaz y que el control por rol funciona como se diseñó.

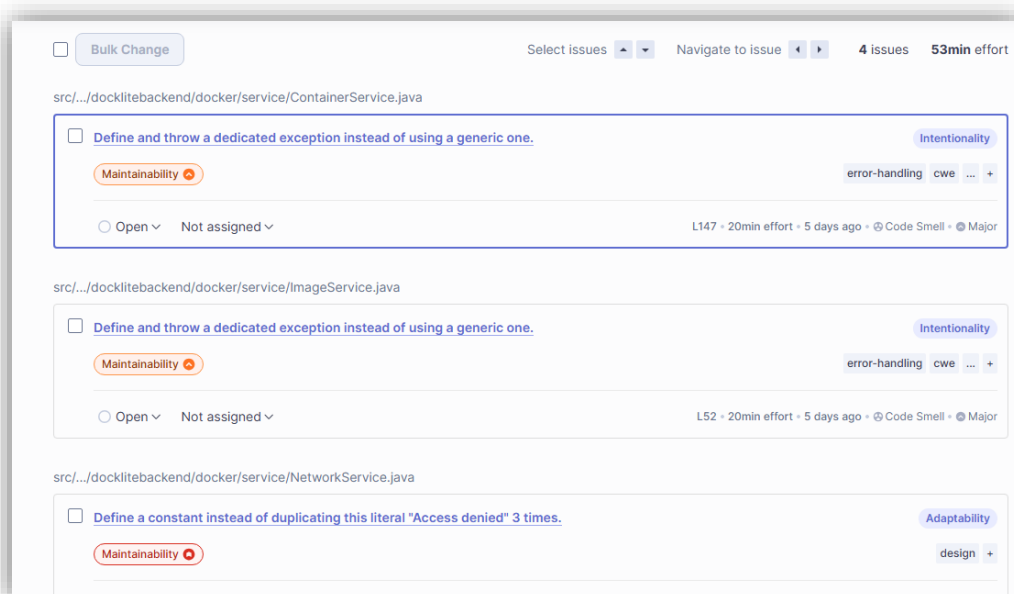
8.4 Análisis estático con SonarQube

A lo largo del desarrollo se ha utilizado SonarQube como herramienta de análisis estático del código fuente del backend. Esta herramienta analiza el código en busca de bugs potenciales, vulnerabilidades de seguridad, code smells (problemas de mantenibilidad) y duplicaciones, y genera un informe con la severidad de cada hallazgo y la regla concreta que se ha violado.



Cada análisis evalúa el proyecto contra el Quality Gate, un conjunto de criterios mínimos que debe cumplir el código para considerarse apto. El uso continuado de SonarQube durante el desarrollo ha permitido detectar problemas en fases tempranas y aplicar correcciones progresivas, manteniendo la calidad del código bajo control durante toda la implementación.

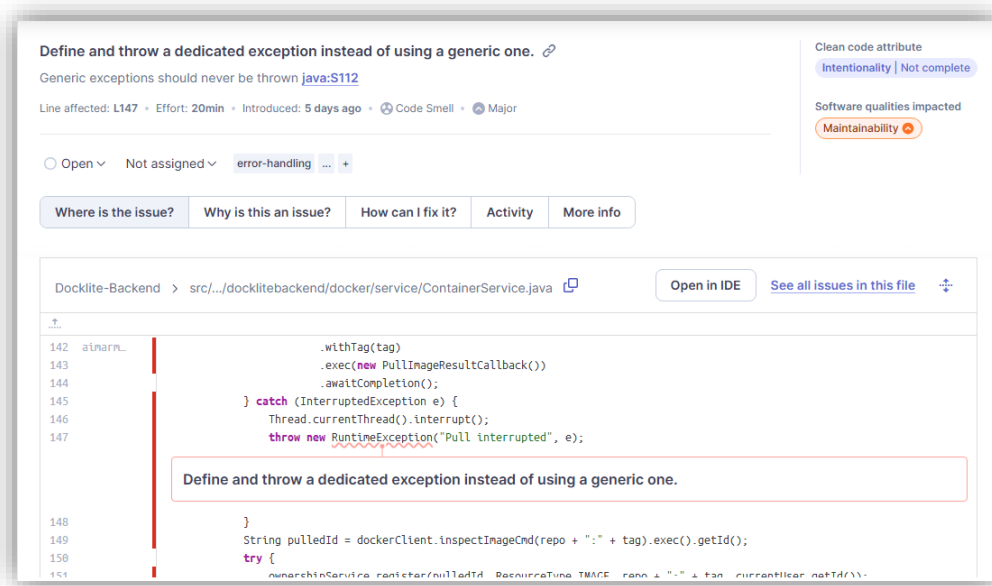
Issues detectadas por SonarQube:



A continuación, se describen algunas de las issues detectadas por SonarQube durante el análisis del proyecto y las correcciones aplicadas:

8.4.1 Excepciones genéricas en operaciones Docker (regla java:S112)

SonarQube detectó otro caso de la misma regla en el servicio encargado de gestionar contenedores, concretamente en el método responsable de garantizar la disponibilidad de una imagen antes de crear el contenedor. En él, una interrupción durante la descarga de la imagen se reenvolvía como una excepción genérica de tipo runtime. Aunque funcionalmente correcto, lanzar una excepción genérica no aporta información sobre qué falló y dificulta el manejo diferenciado de los errores en la capa superior.



La solución consistió en crear una excepción específica del dominio, ubicada junto al resto de excepciones propias de la aplicación. Se sustituyó la excepción genérica de tipo runtime por esta nueva clase, y se aplicó el mismo cambio en el servicio de imágenes, donde se reproducía el mismo patrón al descargar una imagen. Adicionalmente se registró un manejador en el componente global de gestión de excepciones para devolver una respuesta 500 con un mensaje genérico, sin filtrar detalles internos al cliente.

Define and throw a dedicated exception instead of using a generic one. [🔗](#)

Generic exceptions should never be thrown [java:S112](#)

Effort: 20min • Introduced: 5 days ago • Code Smell • Major

Fixed Not assigned error-handling ... +

Where is the issue?

Why is this an issue?

How can I fix it?

Activity

More info

This issue is **Closed (Fixed)**. It was detected in the file below and is no longer being detected.

Docklite-Backend > src/.../docklitebackend/docker/service/ContainerService.java

Open in IDE

1 ainarn... package es.docklite.docklitebackend.docker.service;

2

3 import com.github.dockerjava.api.DockerClient;

4 import com.github.dockerjava.api.async.ResultCallback;

5 import com.github.dockerjava.api.command.CreateContainerResponse;

6 ainarn... import com.github.dockerjava.api.command.PullImageResultCallback;

7 import com.github.dockerjava.api.exception.NotFoundException;

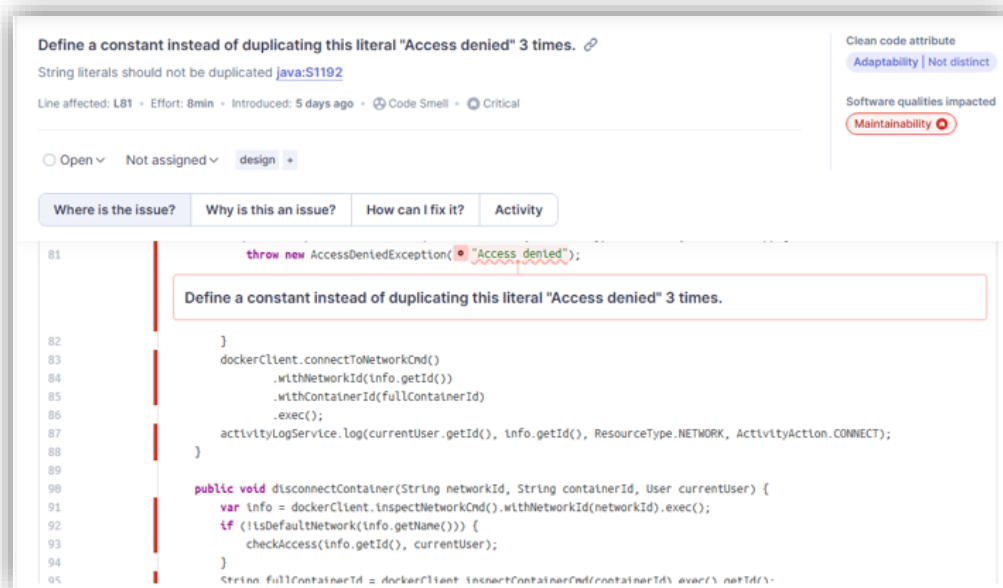
8 ainarn... import com.github.dockerjava.api.model.Container;

9 import com.github.dockerjava.api.model.Frame;

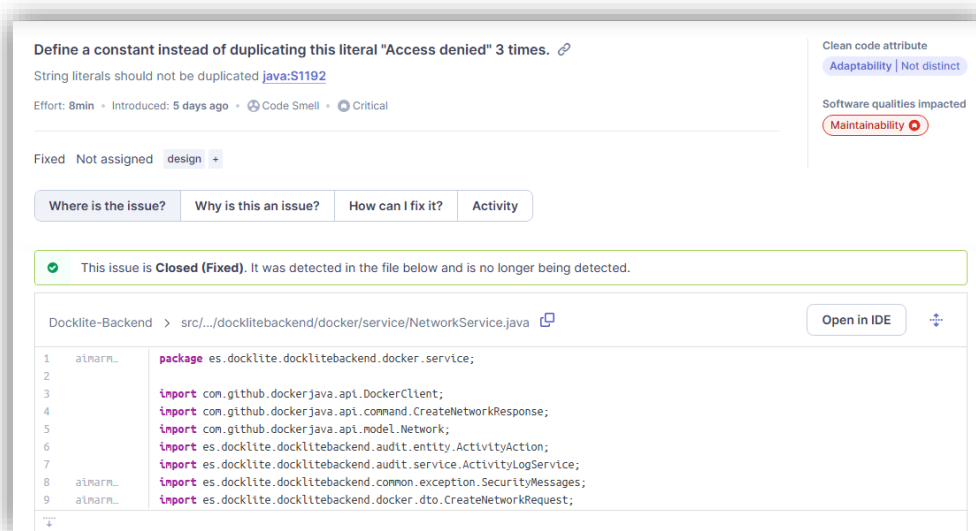
...

8.4.2 Duplicación de literales (regla java:S1192)

SonarQube detectó la repetición del literal "Access denied" tres veces en el servicio de redes, marcándolo como code smell de severidad Critical (categoría Maintainability). La regla recomienda extraer los literales repetidos a una constante para evitar errores de refactorización (cambiar un sitio y olvidar otro) y dejar más clara la intención del código.



La solución consistió en crear una clase de constantes compartida en el paquete de excepciones comunes y centralizar el mensaje en una constante pública estática. Aprovechando el refactor, se sustituyeron también las ocurrencias del mismo literal en otros archivos del proyecto (servicios de contenedores y volúmenes, manejador global de excepciones y manejador de accesos denegados en formato JSON), de modo que todo el código de control de acceso comparte ahora la misma fuente de verdad. Si el mensaje cambiara en el futuro, bastaría con modificarlo en un único sitio.



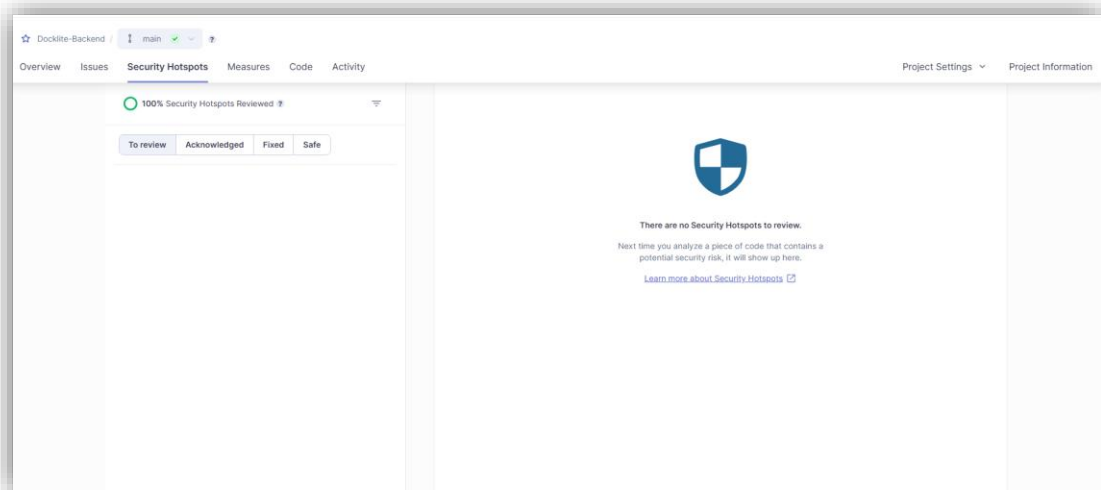
8.4.3 Security Hotspots

Además de los code smells anteriores, SonarQube identifica zonas de código sensibles a la seguridad denominadas Security Hotspots. A diferencia de las issues, los hotspots no son problemas detectados de forma automática, sino fragmentos que requieren una revisión humana para confirmar que su uso es seguro en el contexto concreto de la aplicación.

Docklite presenta un único hotspot, correspondiente a la desactivación de la protección CSRF en la configuración de seguridad. Tras la revisión, se ha marcado como Safe con la siguiente justificación:

La aplicación es una API REST stateless, autenticada exclusivamente mediante JWT en la cabecera Authorization. Al no utilizarse cookies de sesión, los ataques de tipo CSRF no son aplicables, ya que dependen de que el navegador adjunte automáticamente la cookie de sesión a peticiones cross-origin. La propia documentación oficial de Spring Security recomienda desactivar CSRF en escenarios stateless.

Tras este análisis, el Quality Gate del proyecto se mantiene en verde con cero issues abiertas y todos los hotspots revisados.



9 Conclusiones

El desarrollo de Docklite ha cumplido su objetivo principal: ofrecer una interfaz web sencilla para la gestión remota de contenedores Docker, desplegable de forma autónoma en cualquier servidor Linux. Se han implementado los nueve requisitos funcionales y los seis no funcionales definidos al inicio del proyecto.

El resultado es un sistema que permite al administrador controlar el acceso mediante invitaciones multi-uso, garantiza el aislamiento de los recursos Docker entre usuarios y audita las acciones mutantes en una tabla independiente. Todo ello se distribuye mediante un único script de instalación interactivo que automatiza la configuración inicial, incluyendo la obtención de un certificado HTTPS de Let's Encrypt cuando procede.

Durante el desarrollo se han integrado tecnologías ajenas al temario del ciclo, como Spring Boot 4 o la API de *docker-java*, y se han aplicado prácticas profesionales como las migraciones versionadas con *Flyway*, la autenticación JWT, la contenedorización multi-stage o la documentación con OpenAPI. La principal lección extraída es la importancia de pensar el despliegue desde el primer día y de externalizar la configuración sensible mediante variables de entorno.

El proyecto deja abiertas varias líneas de mejora futuras: la incorporación de pruebas automatizadas, el aislamiento del socket Docker mediante un *socket-proxy*, la renovación automática de certificados, el soporte para más distribuciones Linux y la internacionalización de la interfaz.

En conjunto, Docklite se entrega como una aplicación funcional, documentada y lista para producción, dando por cumplido el propósito del Trabajo de Fin de Grado.

10 Bibliografía

10.1 Documentación oficial

Spring Boot Reference Documentation. VMware, Inc.

<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring Security Reference. VMware, Inc.

<https://docs.spring.io/spring-security/reference/>

Spring Data JPA Reference. VMware, Inc.

<https://docs.spring.io/spring-data/jpa/reference/>

Hibernate ORM User Guide. Red Hat, Inc.

[https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate User Guide.html](https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate%20User%20Guide.html)

Flyway Documentation. Redgate Software.

<https://documentation.red-gate.com/flyway>

PostgreSQL 16 Documentation. The PostgreSQL Global Development Group.

<https://www.postgresql.org/docs/16/>

Docker Engine Documentation. Docker, Inc.

<https://docs.docker.com/engine/>

Docker Java Client API. docker-java contributors.

<https://github.com/docker-java/docker-java>

Angular Documentation. Google.

<https://angular.dev/>

nginx Documentation. F5, Inc.

<https://nginx.org/en/docs/>

Let's Encrypt Documentation. Internet Security Research Group.

<https://letsencrypt.org/docs/>

10.2 Especificaciones y estándares

Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519 — JSON Web Token (JWT). IETF.

<https://datatracker.ietf.org/doc/html/rfc7519>

Jones, M. (2015). RFC 7518 — JSON Web Algorithms (JWA). IETF.

<https://datatracker.ietf.org/doc/html/rfc7518>

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine. (Tesis doctoral que define el estilo REST).
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

10.3 Buenas prácticas y seguridad

OWASP Foundation. OWASP Top Ten.
<https://owasp.org/www-project-top-ten/>

OWASP Cheat Sheet Series — Password Storage.
[https://cheatsheetseries.owasp.org/cheatsheets/Password Storage Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

The Twelve-Factor App. Adam Wiggins.
<https://12factor.net/>

10.4 Herramientas

SonarQube Documentation. SonarSource SA.
<https://docs.sonarsource.com/sonarqube/>

Springdoc OpenAPI Documentation.
<https://springdoc.org/>

10.5 Recursos consultados durante el desarrollo

Baeldung. Tutorial: JWT Authentication and Authorization with Spring Security.
<https://www.baeldung.com/spring-security-oauth-jwt>

Baeldung. Spring Boot Multi-Stage Docker Build.
<https://www.baeldung.com/spring-boot-docker-images>